

Mobile Development — Case Studies & Answer Key

Focus: Mobile Threats · Mobile Networks · App Security (Modules 3 & 6) — with one broader scenario

Exam signal: the lecturer said he will set "two case studies covering Mobile Threats, Mobile Networks and App Security." This pack gives **four practice case studies** in that style. **Questions come first (Part 1); the full answer key is at the end (Part 2).**

Part 1 — Case Study Questions

Case Study 1 — "MoMo Fraud" (Mobile Threats & App Security)

Ngwa, a trader in Douala, uses the MTN Mobile Money app. At a café he connects to a free hotspot named "Douala_Free_WiFi" to check his balance. That week he also receives an SMS "from MTN" saying: "Your account is blocked, confirm your PIN here: secure-mtn-momo.cm". Days later his number stops working and money vanishes from his wallet — his SIM had been swapped.

1. Identify and explain **THREE** mobile security threats illustrated in this scenario. **(6)**
2. Explain how a **Man-in-the-Middle (MITM)** attack works on public Wi-Fi, and name the hardware feature most commonly exploited. **(4)**
3. What is **smishing**, and why is it often more effective on mobile than email phishing? **(4)**
4. Recommend **FOUR** security best practices the MoMo developers should implement to protect users. **(4)**
5. Define **SIM-swap fraud**. **(2)**

Case Study 2 — "Coverage on the Move" (Mobile Networks)

A telecom operator is upgrading a region from 3G to 4G/5G. A commuter streams video while driving on the highway, and her call never drops as she moves between towers. In a football stadium, tens of thousands of fans connect at the same time.

1. Describe the **key components** of a mobile network. **(5)**
2. Compare **2G, 3G, 4G and 5G**, stating one key capability each introduced. **(4)**
3. Define **handover** and explain why the commuter's call does not drop. **(3)**
4. Name the **cell types** from largest to smallest, and state which best suits the crowded stadium and why. **(4)**
5. What is the **IMEI** and what is the **SIM**? Differentiate them. **(4)**

Case Study 3 — "HealthTrack Security Audit" (App Security / Secure Development)

A startup builds HealthTrack, a React Native + Firebase app storing patients' medical records. A security audit finds: JWT tokens saved in AsyncStorage, passwords hashed with MD5, an API key hardcoded in the app, Firebase rules set to public, and network traffic sent over HTTP.

1. For **EACH** of the five findings, state the vulnerability and the correct fix. **(10)**
2. Map any **THREE** of these findings to the **OWASP Mobile Top 10**. **(3)**
3. Differentiate **SAST** and **DAST** security testing, naming one tool for each. **(4)**
4. What is **certificate pinning**, and which threat does it defend against? **(3)**

Case Study 4 — "PayGo Super-App Launch" (broader: apps, networks, threats, monetization)

A Cameroonian company is launching PayGo, a super-app for payments, ride-hailing and shopping. They must choose an app-development approach, it will run over 4G/5G, it processes payments (so security is critical), and they need a revenue model.

1. Differentiate **native, hybrid and cross-platform** apps, and recommend one for PayGo with justification. **(6)**
2. As it processes payments, identify **THREE** security threats and a mitigation for each. **(6)**
3. Which **5G** features benefit a real-time ride-hailing service? **(3)**
4. Suggest and justify **TWO** monetization models for PayGo. **(4)**
5. State **TWO** reasons mobile security is uniquely challenging compared to desktop. **(2)**

Case Study 1 — MoMo Fraud

1. Three threats.

- **Insecure communication / MITM** — connecting to the untrusted "Douala_Free_WiFi" hotspot let an attacker intercept his app traffic.
- **Social engineering (Smishing)** — the fake SMS with a link tricking him into revealing his PIN.
- **SIM-swap fraud** — the attacker transferred his number to their own SIM to intercept his OTPs and calls.

(Insecure data storage would also be accepted.)

2. MITM on public Wi-Fi.

The attacker sets up a **rogue access point** — a fake Wi-Fi hotspot with a trusted-sounding name. When the victim connects, all traffic passes through the attacker, who can capture unencrypted data, steal session cookies or redirect to phishing sites. Hardware exploited: the **Wi-Fi radio/adapter**.

3. Smishing.

Phishing delivered via SMS. More effective because: users **trust SMS** more than email; SMS is **viewed immediately**; and mobile browsers **hide/truncate URLs**, making a fake address (secure-mtn-momo.cm) hard to tell from the real one (mtn.cm).

4. Four best practices.

- **HTTPS + certificate pinning** to stop MITM.
- Store tokens/credentials in the **iOS Keychain / Android Keystore**.
- Require **MFA / biometric authentication** for transactions.
- **User education + anomaly detection** for unusual transactions (plus short-lived tokens).

5. SIM-swap fraud.

Convincing or tricking the operator into transferring a victim's phone number onto a SIM the attacker controls, so the attacker receives the victim's calls and SMS one-time passwords.

Case Study 2 — Mobile Networks

1. Key components.

User Equipment (device) · **SIM card** · **Cell tower / base station** · Base Station Controller (BSC) · **Mobile Switching Centre (MSC)** · **Core Network** · **Backhaul links**.

2. Generations.

2G	3G	4G LTE	5G
Digital voice + SMS + encryption (GSM, SIM card)	Mobile broadband (web, video calls)	Ultra-fast broadband / streaming (100 Mbps+)	Ultra-low latency (<1 ms), massive device density, network slicing

3. Handover.

The **seamless transfer** of a connection from one tower to a stronger one as the user moves, without dropping the call. The phone monitors signal strength of nearby towers, and the network switches in milliseconds — so the commuter's call continues.

4. Cell types.

Macrocell > Microcell > Picocell > Femtocell > Small cell (5G). A crowded stadium is best served by many **small cells / picocells** that add capacity in a dense area; 5G **massive MIMO / beamforming** lets one tower serve hundreds of devices at once.

5. IMEI vs SIM.

The **IMEI** is the device's unique identifier — its "digital fingerprint" (identifies the **phone**). The **SIM** (Subscriber Identity Module) identifies the **subscriber** and stores authentication credentials (identifies the **user/account**).

Case Study 3 — HealthTrack Security Audit

1. Five findings — vulnerability & fix.

Finding	Vulnerability	Fix
JWT in AsyncStorage	Insecure token storage (accessible/XSS)	Store tokens in iOS Keychain / Android Keystore
MD5 password hashing	Insufficient / broken cryptography	Use bcrypt, Argon2 or PBKDF2
Hardcoded API key	Secret exposed when app is decompiled	Never hardcode — server-side secrets / env variables
Public Firebase rules	Security misconfiguration (anyone read/write)	Restrict with proper Firebase security rules (auth-based)
HTTP traffic	Insecure communication (MITM)	Enforce HTTPS/TLS everywhere (Network Security Config / ATS)

2. OWASP Mobile Top 10 mapping (any three).

MD5 hashing → **M10 Insufficient Cryptography**; hardcoded API key → **M1 Improper Credential Usage**; public Firebase → **M8 Security Misconfiguration**; JWT in AsyncStorage → **M9 Insecure Data Storage**; HTTP → **M5 Insecure Communication**.

3. SAST vs DAST.

- **SAST** (Static) — analyses **source code without running** the app. Tool: **MobSF** (or SonarQube, Semgrep).
- **DAST** (Dynamic) — tests the **running app**, intercepting traffic/probing APIs. Tool: **Burp Suite** (or OWASP ZAP, Frida).

4. Certificate pinning.

Embedding the server's certificate/public-key hash inside the app so it **only trusts that specific certificate**, not any CA-issued one. It defends against **MITM attacks** that use fraudulent certificates.

Case Study 4 — PayGo Super-App

1. App approaches + recommendation.

- **Native** — platform language (Kotlin/Swift); best performance & hardware access, but two codebases.
- **Hybrid** — web code in a WebView; one cheap codebase, but lower performance.
- **Cross-platform** — one codebase compiling to near-native (React Native, Flutter); ~90% code reuse, good performance.

Recommendation: Cross-platform (React Native/Flutter) — PayGo needs strong performance and security *and* fast delivery on both Android & iOS, which cross-platform balances well. (Pure **native** is also defensible for maximum payment security.)

2. Three payment threats + mitigation.

- **Insecure communication (MITM)** → HTTPS + certificate pinning.
- **Broken authentication** → MFA/biometrics + tokens in Keychain/Keystore.
- **Insufficient cryptography** → AES-256 for data, bcrypt/Argon2 for passwords, hardware-backed keys.

3. 5G features for ride-hailing.

Ultra-low latency (<1 ms) for real-time driver-rider matching and live tracking; **high device density** (many users at once); and **high speed** — enabling lag-free real-time location updates.

4. Two monetization models.

- **Transaction commission** — a small fee on each payment/ride (core revenue for a payments super-app).
- **In-app advertising** — monetize free users via ads. (Also valid: subscription/premium tier, freemium.)

5. Two reasons mobile security is uniquely challenging.

Phones can be **lost or stolen** (physical access); users frequently connect to **insecure public Wi-Fi**. (Also: Android fragmentation/slow patches, sideloading, broad permissions, third-party SDKs.)